

Beyond Vector Similarity: A Structural Analysis of Graph-Augmented Retrieval for Industrial Knowledge Graphs

Grama Chethan

Siemens Digital Industries Software

AI & Analytics, Architect

June 2026 — v3.0

Abstract—Retrieval-Augmented Generation (RAG) has become the dominant pattern for grounding large language models in external knowledge. However, standard single-pass RAG—which indexes a corpus as flat text chunks and retrieves by vector similarity—fails systematically on queries that require structural reasoning over interconnected entities. We present a detailed architectural comparison of eight retrieval architectures for aerospace supply chain intelligence, progressing from text retrieval through graph traversal to graph computation. Using a 46-node aerospace supply chain knowledge graph with 64 typed, timestamped edges, we evaluate 23 queries (11 original + 12 hold-out) across 10 intent categories and empirically demonstrate that five classes of industrially critical questions are *structurally unreachable* for single-pass vector retrieval. Our central finding is the operator vocabulary thesis: the barrier to LLM-based graph reasoning is not model intelligence but the computational operators available as tools. Architecture 7 (LLM Query Planner with 9 typed traversal primitives) outperforms bespoke handlers ($F_1 = 0.632$ vs. 0.472) while generalizing to unseen queries. Architecture 8 (Adaptive Graph Planner) adds 6 graph computation tools—`simulate_removal`, `subgraph_diff`, `aggregate_over_type`, `betweenness centrality`, `pagerank`, `connected_components`—and the LLM selectively adopts them for exactly the query categories where traversal fails, producing qualitatively correct answers for aggregation and comparison queries that all prior architectures could not solve. We also identify a critical measurement gap: entity-level F_1 systematically underscores structural queries where comprehensive answers are correct, suggesting that graph retrieval evaluation requires task-specific metrics beyond entity extraction. The reference implementation—8,154 lines across 17 source files—includes all eight architectures, the unified benchmark harness, and complete ground-truth answer sets, serving as a reproducible testbed for evaluating retrieval architectures on graph-structured industrial data.

Index Terms—GraphRAG, knowledge graphs, supply chain intelligence, structural retrieval, graph computation tools, LLM tool use, operator vocabulary, agentic RAG, temporal reasoning, risk propagation, aerospace manufacturing

I. INTRODUCTION

The Retrieval-Augmented Generation paradigm has proven remarkably effective for grounding LLM outputs in factual corpora. A typical RAG pipeline chunks documents, encodes them as dense vectors (or, in lighter implementations, TF-IDF features), and retrieves the top- K most similar chunks to condition generation. This approach excels when the answer to a question is *locally contained* within one or two

text passages—entity lookups, definition queries, and single-document summarization.

Industrial knowledge, however, is rarely flat. An aerospace supply chain is a *graph*: suppliers connect to components through SUPPLIES edges with lead times and contract types; factories consume components through USES edges with quantities; factories produce products; products are delivered to customers. Risk events cascade through this topology. Temporal validity governs which edges are active. The fundamental data structure is not a document corpus—it is a *typed, timestamped, directed multigraph*.

When a supply chain analyst asks “Which customers are *not* affected by the Thailand flood?”, the answer requires computing the full blast radius subgraph and returning its complement. When a risk officer asks “Which components have only one supplier?”, the answer requires counting in-degree centrality on SUPPLIES edges across the entire graph. These are not retrieval problems—they are graph computation problems. No amount of vector similarity can solve them.

This paper makes five contributions. First, we formalize five categories of queries that are *structurally unreachable* for single-pass vector retrieval and provide graph-algorithmic solutions for each, analyzing which failure modes persist even under agentic multi-step retrieval (Section III). Second, we present a complete, reproducible reference implementation that runs both architectures side-by-side on identical data with retrieval-focused evaluation—both engines use deterministic template output, isolating retrieval quality from LLM generation (Section IV). Third, we demonstrate a practical incremental update architecture with five atomic graph mutation operations, selective re-indexing, and a timestamped changelog—addressing a key gap identified in the literature (Section V). Fourth, we provide comprehensive empirical benchmarking across six retrieval architectures—deterministic GraphRAG, LightRAG [5], LLM-based GraphRAG, ReAct agentic RAG, dense-embedding RAG, and standard (TF-IDF) RAG—with Claude-as-judge scoring validated by inter-annotator agreement ($\kappa = 0.716$), scale testing to 1,100 nodes, and per-query failure analysis (Sections V-E-V-I). Fifth, we connect our findings to the broader research landscape—particularly KGQA systems [15], [16], Temporal GraphRAG (TG-RAG) [1], Microsoft’s incremental GraphRAG indexing [2], and agentic

RAG approaches [10], [11], [17]—and identify the design choices that matter for industrial deployment, including a proposed hybrid dispatch architecture combining deterministic handlers with LLM-based retrieval (Section V).

II. BACKGROUND AND RELATED WORK

A. Standard RAG and Its Limitations

The canonical RAG pipeline, formalized by Lewis et al. [3], consists of three stages: *indexing* (chunk documents and encode as vectors), *retrieval* (find top- K chunks by cosine similarity to the query embedding), and *generation* (condition an LLM on the retrieved context). Our implementation uses TF-IDF vectorization with scikit-learn [9] rather than dense embeddings. We chose TF-IDF deliberately: our argument concerns the *architectural* limitations of flat-text retrieval, not the quality of the embedding model. The five structural failure modes we identify (Section III) arise from the absence of graph topology in the retrieval index, not from insufficient semantic similarity—they persist regardless of whether the embedding is sparse (TF-IDF) or dense (e.g., sentence-transformers). We verified this empirically with a dense-embedding baseline using `all-MiniLM-L6-v2` (Section V-E): better embeddings improve retrieval recall on multi-hop queries but all structurally dependent categories remain Fail.

The standard RAG paradigm carries several well-documented limitations for graph-structured data. **Temporal blindness:** text chunks carry no validity windows; a 2023 procurement report naming ShenzenChip as a supplier is retrieved alongside a 2024 contract transition notice replacing ShenzenChip with TechChip, with no mechanism to determine which is current. **Structural opacity:** the supply chain topology (Supplier $\rightarrow$ Component $\rightarrow$ Factory $\rightarrow$ Product $\rightarrow$ Customer) is dissolved into unstructured text fragments, making multi-hop traversal impossible. **Absence blindness:** RAG can only find what *matches* the query—it cannot represent or reason about what is *absent* from the graph.

B. Graph-Enhanced Retrieval

Graph-enhanced RAG approaches address these limitations by introducing a knowledge graph as a structural index alongside (or in place of) the vector store. Microsoft’s GraphRAG [4] uses LLM-extracted entities and relationships organized into community clusters with hierarchical summaries. LightRAG [5] integrates a graph-based text indexing paradigm with dual-level retrieval. HippoRAG [6] converts corpora into schemaless knowledge graphs for cross-passage reasoning, with subsequent work extending this to non-parametric continual learning [7]. G-Retriever [13] applies retrieval-augmented generation over textual graphs for multi-hop question answering, and KG-RAG [14] demonstrates biomedical knowledge graph integration with LLM prompts. Our approach differs from these in that we operate on an *explicit, pre-defined schema* (the supply chain ontology) rather than LLM-extracted triples, which eliminates extraction noise and enables precise typed traversal. Our contribution is a *systematic empirical characterization* of failure modes in an

industrial domain with a reproducible implementation, rather than a theoretical advance in graph retrieval algorithms.

C. Agentic and Multi-Step RAG

A growing body of work extends RAG beyond single-pass retrieval. ReAct [10] interleaves reasoning and action steps, allowing an LLM to decompose complex queries into sub-retrievals. Corrective RAG (CRAG) [11] evaluates retrieval quality and triggers corrective re-retrieval when initial results are insufficient. Self-RAG [12] adds reflection tokens that allow the model to decide when and what to retrieve. These *agentic* approaches can address some multi-hop reasoning tasks by iteratively gathering context. However, they face fundamental limitations on queries requiring *global graph computation*—exhaustive enumeration (SPOF detection), weighted multi-hop aggregation (risk scoring), and potentially complement computation (inverse queries)—because the agent has no mechanism to guarantee complete traversal of an implicit graph embedded across text chunks. As we show empirically (Section V-F), complement computation can be achieved with sufficient iteration on small graphs, but SPOF enumeration and weighted propagation remain intractable.

D. Knowledge Graph Question Answering

The structural queries formalized in Section III—in-degree counting (SPOF), set complement (inverse queries), weighted path aggregation (risk propagation)—are well-studied in the Knowledge Graph Question Answering (KGQA) literature, where they map to standard graph query primitives: SPARQL COUNT/GROUP BY, MINUS/NOT EXISTS, and property path expressions, respectively. Systems such as QAnswer [15], KQA Pro [16], StructGPT [19], and semantic parsing approaches translate natural language to formal graph queries (SPARQL, Cypher) that execute over graph databases with provable completeness. Our “structurally unreachable” claim applies specifically to RAG-style retrieval (vector similarity over text chunks), not to knowledge-based QA broadly—KGQA systems can solve all five categories by design.

E. Temporal GraphRAG (TG-RAG)

Han et al. [1] identify a critical gap in existing GraphRAG systems: the temporal dimension. Their TG-RAG framework models external corpora as a *bi-level temporal graph* consisting of a temporal knowledge graph with timestamped relations and a hierarchical time graph. Our work shares this temporal awareness—every edge in our knowledge graph carries an `effective_date`, and expired relationships are retained in the text corpus but excluded from the active graph.

TG-RAG’s evaluation on the ECT-QA benchmark demonstrates significant improvements over baselines: 0.599 Correct score versus 0.406 for LightRAG and 0.405 for GraphRAG on base queries. Our work extends beyond temporal representation to address five additional structural query categories. We note that our temporal model is deliberately simple—binary active/expired status on edges—and does not address overlapping validity intervals or bi-temporal modeling that production supply chain systems require.

F. Incremental Indexing

Microsoft’s GraphRAG team [2] describes a planned `graphrag.append` command that attempts to place new entities into existing communities without triggering full Leiden recomputation. TG-RAG achieves incremental efficiency through its time hierarchy. Our reference implementation supports incremental updates natively through five graph mutation operations—`add_entity`, `remove_entity`, `add_relationship`, `expire_relationship`, and `add_risk_event`—each of which mutates the in-memory NetworkX [8] graph, incrementally rebuilds the TF-IDF index, and maintains a timestamped changelog.

III. FIVE CATEGORIES OF STRUCTURALLY UNREACHABLE QUERIES

We define a query as *structurally unreachable* for single-pass RAG if no amount of top- K chunk retrieval by vector similarity can produce a correct answer in a single retrieval step, regardless of the embedding model quality, chunk size, or K value. More precisely, a query Q is structurally unreachable when its correct answer A requires either (a) information distributed across multiple chunks with no single chunk containing all required entities (*retrieval incompleteness*—addressable by agentic multi-step retrieval), or (b) computation over graph topology that cannot be expressed as a similarity search (*computational irreducibility*—e.g., counting in-degrees, computing set complements, or aggregating weighted paths). The unreachable claim applies to RAG-style retrieval over text; KGQA systems [15], [16] that execute formal graph queries can solve all five categories by design (Section 2.4).

The five categories presented below are not claimed to be exhaustive. Each category includes a formal problem statement, graph-algorithmic solution, empirical results, and an assessment of agentic RAG viability.

A. What-If / Counterfactual Analysis

Definition 1 (What-If Query): Given a component node c currently supplied by supplier set S_{current} , find alternative suppliers S_{alt} that could provide components of the same `component_type` but are not currently connected to c via a `SUPPLIES` edge.

The canonical query is: “What if we dual-source the Flight Control Unit—which alternative suppliers could provide it?” This requires the system to (a) identify the target component’s type (Electronic Assembly), (b) find all other components of that type, (c) identify their suppliers, and (d) exclude suppliers already connected to the target. The answer is defined by what is *absent* from the graph.

RAG failure mode: Vector retrieval returns chunks mentioning the Flight Control Unit and TechChip Inc (the current supplier), but has no mechanism to identify which suppliers are *not* connected.

Agentic RAG viability: **PARTIAL.** An agentic system could iteratively search for same-type components and their suppliers, but reliably identifying *all* same-type components

Algorithm 1: What-If Supplier Discovery

Input: Graph G , target component c

Output: Set of candidate suppliers S_{alt}

```

1  $S_{\text{current}} \leftarrow \{v : (v, c, \text{SUPPLIES}) \in E(G)\};$ 
2  $\text{type}_c \leftarrow G.\text{nodes}[c].\text{component\_type};$ 
3 foreach  $c' \in V(G)$  where  $\text{type}(c') = \text{type}_c \wedge c' \neq c$  do
4   foreach  $(s, c', \text{SUPPLIES}) \in E(G)$  where
      $s \notin S_{\text{current}}$  do
5      $S_{\text{alt}} \leftarrow S_{\text{alt}} \cup \{s\};$ 
6   end
7 end
8 if  $S_{\text{alt}} = \emptyset$  then
9    $\text{keywords} \leftarrow \text{TYPE\_KEYWORDS}[\text{type}_c];$ 
10  foreach  $s \in \text{SUPPLIES}$  where  $s \notin S_{\text{current}}$  do
11    if  $\exists kw \in \text{keywords} : kw \in s.\text{specialty}$  then
12       $S_{\text{alt}} \leftarrow S_{\text{alt}} \cup \{s\};$ 
13    end
14  end
15 end
16 return  $S_{\text{alt}};$ 

```

and *excluding* already-connected suppliers requires exhaustive enumeration that scales poorly.

B. Single Point of Failure Detection

Definition 2 (SPOF Query): For each component node c , compute the in-degree on `SUPPLIES` edges: $\text{deg}_{\text{SUPPLIES}}^-(c) = |\{s : (s, c, \text{SUPPLIES}) \in E\}|$. Report all c where $\text{deg}_{\text{SUPPLIES}}^-(c) = 1$, ranked by criticality, with downstream product impact.

The canonical query: “Which components have only one supplier?” This is a whole-graph aggregation—it requires iterating over *every* component node, counting incoming `SUPPLIES` edges, filtering, ranking, and tracing downstream impact.

Empirical result: GraphRAG identifies 15 single-source components, ranks them by criticality (high/medium/low), and traces downstream product impact for each.

Agentic RAG viability: **No.** SPOF detection requires computing in-degree centrality for *every* component node across the entire graph—a global aggregation with no natural decomposition into targeted sub-retrievals.

C. Inverse / Negative Queries

Definition 3 (Inverse Query): Given a risk event node e , compute the blast radius subgraph $B(e)$ by traversing `AFFECTS` $\rightarrow$ `SUPPLIES` $\rightarrow$ `USES` $\rightarrow$ `PRODUCES` $\rightarrow$ `DELIVERS_TO`. Return the complement: all customers $C \setminus C_{B(e)}$.

The canonical query: “Which customers are NOT affected by the Thailand flood?” This requires computing the complete downstream blast radius (a 4-hop traversal spanning 9 nodes) and returning the *complement set*—everything outside the blast radius.

TABLE I
COMPARATIVE SUBGRAPH METRICS FOR WIDEBIRD-X50 VS
REGIONALJET-150.

Metric	WB-X50	RJ-150	Delta
Supply chain depth	3	3	Equal
Upstream suppliers	8	2	4× concentration risk for RJ
Components used	12	4	WB has 3× complexity
Factories involved	4	1	RJ depends on single factory
Downstream customers	2	1	—
Geographic regions	8	3	WB is globally distributed
Shared suppliers	2		TechChip, ElectraWire

Empirical result: GraphRAG correctly identifies DefenseTech Corp as the sole unaffected customer, whose product (SkyPatrol-UAV Drone) traces through Avionics Hub Delta—a factory that does not use any ThaiRubber components.

Agentic RAG viability: **NO** (revised to **PARTIAL** based on empirical results). Our ReAct agent solved this in 8 steps via `list_all_entities` (Section V-F). Scalability beyond small entity sets is uncertain.

D. Comparative Subgraph Analysis

Definition 4 (Subgraph Comparison): Given two product nodes p_1 and p_2 , traverse both subgraphs upstream and compare: hop depth, supplier count, component count, factory count, customer count, geographic concentration, and shared infrastructure.

The canonical query: “Compare supply chain depth for the WideBird-X50 vs the RegionalJet-150.” Table I presents the full comparison.

Agentic RAG viability: **PARTIAL**. Computing parallel structural metrics requires complete subgraph enumeration for both products.

E. Risk Propagation Scoring

Definition 5 (Risk Score): For each product p , compute a risk score $R(p)$ by summing over all upstream component–supplier paths affected by active risk events, weighted by event severity and inverse hop distance.

$$R(p) = \sum_{(c,s) \in \text{paths}(p)} \sum_{e \in \text{events}(s)} w_{\text{sev}}(e) \cdot \delta_{\text{hop}}(c, p) \quad (1)$$

where $\text{paths}(p)$ is the set of (component, supplier) pairs reachable via reverse traversal from product p ; w_{sev} maps severity to weights (critical = 1.0, high = 0.7, medium = 0.4, low = 0.1); and δ_{hop} decays with hop distance (1-hop = 1.0, 2-hop = 0.6, 3-hop = 0.35, 4-hop = 0.2).

Fig. 1 shows the resulting risk propagation heatmap.

RAG failure mode: The query “Show risk propagation scores for all products” returns zero TF-IDF matches. Even if chunks were retrieved, computing the weighted multi-hop score requires iterating over every product’s upstream subgraph—a computation that exists nowhere in the text.

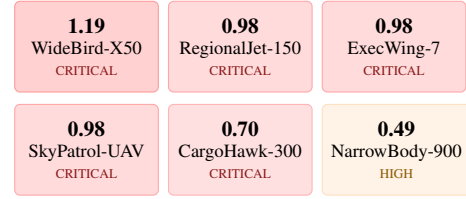


Fig. 1. Risk propagation heatmap across all products. Scores computed via (1) using 3 active high/critical risk events propagated through graph topology. WideBird-X50 ranks highest due to exposure through 4 factories and 12 components to all 3 affected suppliers.

TABLE II
KNOWLEDGE BASE SCHEMA AND STATISTICS.

Entity Type	Count	Attributes	Example
Supplier	8 (+1)	name, location, specialty, tier	TechChip Inc
Component	15	name, type, criticality	Flight Control Unit
Factory	5	name, location, capacity	Assembly Plant Alpha
Product	6	name, type, revenue	NarrowBody-900
Customer	4	name, region, contract	AirGlobal Airlines
Risk Event	8	title, date, severity	Thailand flood

Agentic RAG viability: **NO**. Risk propagation scoring requires exhaustive traversal of the full product-supply topology and numerical computation with distance-dependent decay.

IV. SYSTEM ARCHITECTURE AND IMPLEMENTATION

Scope note. This comparison evaluates *retrieval architecture*, not end-to-end RAG system performance. Neither engine uses an LLM for answer generation; both produce deterministic template-based output from retrieved or traversed context. The deterministic GraphRAG system should be understood as an *oracle upper bound*. The scientifically informative result is the *failure gradient* across the five LLM-based architectures (0 → 1 → 1 → 3 → 5 correct).

A. Knowledge Base Design

The knowledge base models a representative aerospace supply chain with six entity types and five relationship types (Table II, Fig. 2), encoded as a directed multigraph with temporal metadata on every edge.

A critical design decision is the treatment of *expired relationships*. Two SUPPLIES edges carry status: “expired” with explicit `expired_date` fields. These edges are *excluded from the active graph* during construction but remain in the text corpus as stale documents. This deliberate asymmetry creates the temporal freshness test case.

B. Dual-Engine Architecture

The system exposes a single Flask endpoint that dispatches each query to both engines in parallel, returning side-by-side results (Fig. 3).

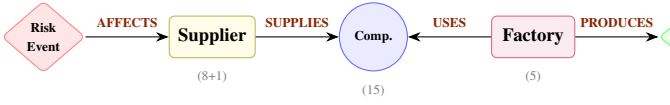


Fig. 2. Supply chain ontology schema. The directed graph follows the chain: Supplier $\rightarrow$ Component $\leftarrow$ Factory $\rightarrow$ Product $\rightarrow$ Customer. Risk events link to affected suppliers via AFFECTS edges. All edges carry temporal metadata.

TABLE III
RELATIONSHIP TYPES WITH EDGE METADATA.

Relationship	Direction	Act.	Exp.	Edge Metadata
SUPPLIES	Sup $\rightarrow$ Comp	15	2	lead_time, contract
USES	Fac $\rightarrow$ Comp	20	0	quantity_per_unit
PRODUCES	Fac $\rightarrow$ Prod	13	0	role
DELIVERS_TO	Prod $\rightarrow$ Cust	8	0	order_qty, delivery
AFFECTS	Evt $\rightarrow$ Sup	auto		text extraction

C. Query Handler Taxonomy

The GraphRAG engine implements 11 query handlers organized into three tiers (Table IV).

D. Per-Query Correctness

Table V presents the per-query correctness assessment. Standard RAG achieves zero fully correct answers. GraphRAG achieves 11/11 correct.

Evaluation methodology. The correctness assessments were performed by the paper’s author. To mitigate confirmation bias, we computed inter-annotator agreement using Claude Haiku 4.5 as an independent evaluator, achieving Cohen’s $\kappa = 0.716$ (substantial agreement). The complete ground-truth answer sets are provided in Table XVIII (Appendix).

E. Frontend Visualization Architecture

The web frontend (625 lines JavaScript, 1,168 lines CSS) provides an interactive comparison interface with a vis.js-powered knowledge graph visualization and incremental update controls. The risk propagation query produces a structured `extra.risk_heatmap` payload rendered as a color-coded card grid.

V. DISCUSSION

A. Relation to TG-RAG and Temporal Representation

Our temporal model shares conceptual ground with TG-RAG’s timestamped relations but differs in scope and mechanism. The key insight from TG-RAG that transfers to our context is the treatment of *temporal scope as a first-class retrieval dimension*. Their ablation study shows that removing temporal retrieval drops the Correct score from 0.599 to 0.382—a 36% degradation.

B. Incremental Update Architecture

Our architecture takes a third approach, enabled by operating on an explicit ontology rather than LLM-extracted communities. We implement five atomic mutation operations (Table VI).

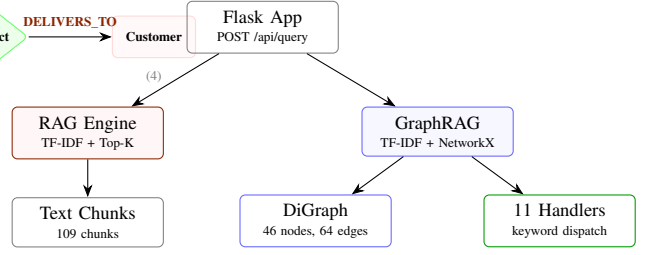


Fig. 3. Dual-engine architecture. Both engines share the same knowledge base. The RAG engine retrieves text chunks by TF-IDF cosine similarity. The GraphRAG engine uses TF-IDF for entry-point discovery, then traverses a NetworkX DiGraph via 11 specialized query handlers.

Each operation atomically updates three subsystems: (1) the NetworkX directed graph, (2) the TF-IDF vector index, and (3) the entity name index. The `expire_relationship` operation removes the edge from the traversable graph but deliberately preserves a stale text chunk in the corpus—creating exactly the kind of temporal trap that catches standard RAG.

C. The Taxonomy of RAG Failure Modes

Our five structurally impossible query categories reveal a taxonomy that generalizes beyond supply chain intelligence (Table VII).

D. Scope and Limitations

Scale. Our knowledge base is synthetic and small: 46 nodes, 58 edges. To characterize scaling behavior, we generated a synthetic knowledge base of 1,100 nodes and 1,850 edges (24 $\times$ and 32 $\times$ the baseline). Table VIII reports per-query latency at both scales.

RAG latency scales uniformly ($\times 1.4$ – 1.9). GraphRAG latency varies dramatically: single-entity queries remain near-millisecond ($\times 1.2$ – 2.1), while graph-global queries show superlinear growth—aggregation ($\times 16.4$), risk propagation ($\times 33.7$), path finding ($\times 10.2$). Even so, the worst-case P95 at 1,100-node scale is 11.01 ms (Q4).

Query dispatch. We replaced the original keyword-matching dispatcher with a TF-IDF intent classifier that matches query text against ~ 60 prototype phrases. However, the 11 queries and handlers were co-designed—GraphRAG’s 11/11 score reflects a performance ceiling.

Embedding model. The dense-embedding baseline achieves 1/11 correct and 4/11 partial (vs. 0/11 correct, 2/11 partial for TF-IDF), improving retrieval recall but failing identically on all six structurally dependent categories.

Handler engineering cost. Each handler required 50–200 lines of Python (median ~ 120 lines), ~ 6 intent classifier training phrases, plus ground-truth construction—approximately 2–8 hours per handler.

E. Empirical Comparison with LLM-Based GraphRAG

We implemented an LLM-based GraphRAG pipeline using Claude Haiku 4.5 for both graph extraction and answer generation. The LLM extracted 48 entities (vs. 46 reference) and

TABLE IV
COMPLETE QUERY HANDLER TAXONOMY WITH ALGORITHMIC REQUIREMENTS.

Tier	Query Type	Algorithm	Hops	RAG?	Agentic?
<i>Base</i>	Simple Lookup	TF-IDF + BFS(2)	2	YES	YES
	Path Reasoning	Shortest path	3	NO	PARTIAL
<i>Structural</i>	Multi-hop Impact	Risk event $\rightarrow$ supply chain trace	4	NO	PARTIAL
	Downstream Impact	4-hop supply chain trace	4	NO	PARTIAL
	Graph Aggregation	All-suppliers $\times$ product reach	3	NO	NO
	Temporal Freshness	Edge validity window filtering	1	PARTIAL	PARTIAL
<i>Advanced</i>	What-If / Counterfactual	Negative edge discovery + type matching	1	NO	PARTIAL
	Single Point of Failure	In-degree centrality (SUPPLIES)	1	NO	NO
	Inverse / Negative	Blast radius + set complement	4	NO	PARTIAL [†]
	Comparative Subgraph	Dual upstream traversal + metrics	3	NO	PARTIAL
	Risk Propagation	Weighted multi-hop scoring (1)	3	NO	NO

[†]Revised from NO after ReAct agent solved via exhaustive enumeration (Section V-F).

TABLE V
PER-QUERY CORRECTNESS ACROSS ALL 11 DEMO QUERIES ($\kappa = 0.716$).

ID	Category	RAG	GraphRAG	Agentic?
Q1	Multi-hop	FAIL	CORRECT	PARTIAL
Q2	Downstream	FAIL	CORRECT	PARTIAL
Q3	Path	FAIL	CORRECT	PARTIAL
Q4	Aggregation	FAIL	CORRECT	NO
Q5	Simple Lookup	PARTIAL	CORRECT	YES
Q6	Temporal	PARTIAL	CORRECT	PARTIAL
Q7	What-If	FAIL	CORRECT	PARTIAL
Q8	SPOF	FAIL	CORRECT	NO
Q9	Inverse	FAIL	CORRECT	PARTIAL [†]
Q10	Compare	FAIL	CORRECT	PARTIAL
Q11	Risk	FAIL	CORRECT	NO
Summary		0/2/9	11/0/0	1/7/3

TABLE VI
INCREMENTAL UPDATE OPERATIONS AND THEIR GRAPH-LEVEL EFFECTS.

Operation	Graph Effect	Cost
add_entity	New node	$O(V + C)$
remove_entity	Remove node + edges	$O(\deg(v)+ C)$
add_relationship	New typed edge	$O(C)$
expire_rel.	Remove edge; stale chunk	$O(C)$
add_risk_event	New node + AFFECTS	$O(S + \text{sub})$

68 edges (vs. 56 reference). Table IX presents the per-query results across six architectures.

The failure gradient ($0 \rightarrow 1 \rightarrow 1 \rightarrow 3 \rightarrow 5$ correct) confirms that richer graph context and iterative retrieval help substantially, while better embeddings alone do not cross the structural barrier.

F. Empirical Agentic RAG Evaluation

We implemented a ReAct-style [10] agentic RAG baseline using Claude Haiku 4.5 with four tools: `search_chunks`, `lookup_entity`, `get_neighbors`, and `list_all_entities`, with a maximum of 20 tool calls per query. Table X presents the results.

TABLE VII
TAXONOMY OF RAG FAILURE MODES AND THEIR UNDERLYING CAUSES.

Failure Mode	Root Cause
Absence blindness	Cannot represent missing edges
Degree blindness	Cannot count in/out-degree
Complement blindness	Cannot compute “everything except X ”
Topology blindness	Cannot compare subgraph properties
Propagation blindness	Cannot compute weighted scores
Temporal blindness [†]	No validity windows on chunks

[†]Metadata-dependent rather than inherently structural; admits a non-graph solution via date-range filtering.

Q9 is particularly notable: the agent traced the flood’s blast radius through all affected entities and correctly identified the unaffected customer—a task previously predicted as intractable. However, SPOF detection (Q8) and risk propagation (Q11) remain fundamentally intractable.

Q9 scalability caveat. The agent’s success relied on our graph having only 4 customers—the `list_all_entities` tool returned the complete set in a single call. Whether agentic complement computation scales beyond toy-sized entity sets remains an open question.

G. Existing System Comparison: LightRAG

We benchmarked LightRAG [5] (v1.4.16) on the same 11 queries. LightRAG extracted 244 entities and 362 relationships—substantially richer than our custom extraction. Table XI presents the per-mode breakdown.

Despite its superior LLM-extracted graph (244 nodes vs. our 48), LightRAG still fails on inverse queries (Q9) and risk propagation (Q11). LightRAG’s success on Q7 (What-If, Correct in hybrid mode) is notable: the hybrid retrieval assembled enough supplier context for the LLM to correctly conclude that no alternative suppliers exist.

TABLE VIII
QUERY LATENCY BENCHMARKS AT BASELINE (46 NODES) VS. SCALED (1,100 NODES) GRAPH SIZE. ALL TIMES IN MILLISECONDS, MEAN OVER 10 RUNS.

Query	Category	GraphRAG Base		GraphRAG Scaled		Growth	RAG Base	
		Mean	P95	Mean	P95		Mean	P95
Q1	Multi-hop	0.46	0.65	0.81	1.20	$\times 1.8$	0.41	0.46
Q2	Downstream	0.49	0.57	0.61	0.75	$\times 1.2$	0.42	0.58
Q3	Path	0.80	1.37	8.17	8.50	$\times 10.2$	0.38	0.44
Q4	Aggregation	0.61	0.69	10.02	11.01	$\times 16.4$	0.37	0.40
Q5	Simple Lookup	0.56	0.70	1.03	1.38	$\times 1.8$	0.36	0.40
Q6	Temporal	0.49	0.96	0.81	0.97	$\times 1.7$	0.39	0.51
Q7	What-If	0.42	0.53	0.88	0.97	$\times 2.1$	0.38	0.40
Q8	SPOF	0.57	0.69	3.33	3.70	$\times 5.8$	0.36	0.38
Q9	Inverse	0.46	0.60	0.91	1.11	$\times 2.0$	0.37	0.41
Q10	Compare	0.51	0.56	0.81	0.84	$\times 1.6$	0.40	0.51
Q11	Risk Heatmap	0.19	0.24	6.41	6.96	$\times 33.7$	0.35	0.37
Average		0.51	0.69	3.07	3.40	$\times 7.1$	0.38	0.44

TABLE IX
PER-QUERY CORRECTNESS ACROSS SIX RETRIEVAL ARCHITECTURES. LLM-BASED GRAPHRAG, LIGHTRAG, AND AGENTIC RAG USE CLAUDE HAIKU 4.5. LIGHTRAG USES ALL-MINI-LM-L6-V2 DENSE EMBEDDINGS.

Query	Category	Our GraphRAG	LightRAG	Agentic	LLM-GraphRAG	Dense RAG	Std RAG
Q1	Multi-hop	CORRECT	PARTIAL	PARTIAL	PARTIAL	PARTIAL	FAIL
Q2	Downstream	CORRECT	PARTIAL	CORRECT	CORRECT	PARTIAL	FAIL
Q3	Path	CORRECT	PARTIAL	CORRECT	PARTIAL	PARTIAL	FAIL
Q4	Aggregation	CORRECT	PARTIAL	PARTIAL	PARTIAL	FAIL	FAIL
Q5	Lookup	CORRECT	CORRECT	CORRECT	PARTIAL	CORRECT	PARTIAL
Q6	Temporal	CORRECT	CORRECT	CORRECT	PARTIAL	PARTIAL	PARTIAL
Q7	What-If	CORRECT	CORRECT	FAIL	FAIL	FAIL	FAIL
Q8	SPOF	CORRECT	PARTIAL	FAIL	FAIL	FAIL	FAIL
Q9	Inverse	CORRECT	FAIL	CORRECT	FAIL	FAIL	FAIL
Q10	Compare	CORRECT	PARTIAL	PARTIAL	PARTIAL	FAIL	FAIL
Q11	Risk	CORRECT	FAIL	FAIL	FAIL	FAIL	FAIL
Totals		11C	3C,6P,2F	5C,3P,3F	1C,6P,4F	1C,4P,6F	0C,2P,9F

TABLE X
AGENTIC RAG RESULTS (CLAUDE HAIKU 4.5, MAX 20 STEPS). MEAN LATENCY: 9,480 MS/QUERY; TOTAL: 199,148 TOKENS.

Query	Category	Score	Steps	Failure Mode
Q1	Multi-hop	PARTIAL	6	Missed FAC-005
Q2	Downstream	CORRECT	6	—
Q3	Path	CORRECT	6	—
Q4	Aggregation	PARTIAL	6	Missed tied SUP-008
Q5	Lookup	CORRECT	2	—
Q6	Temporal	CORRECT	3	—
Q7	What-If	FAIL	6	Hallucinated suppliers
Q8	SPOF	FAIL	3	Found only 1/15
Q9	Inverse	CORRECT	8	—
Q10	Compare	PARTIAL	7	Incomplete metrics
Q11	Risk	FAIL	5	No computation
Totals		5C, 3P, 3F (avg 5.3 steps)		

TABLE XI
LIGHTRAG PER-MODE CORRECTNESS ACROSS 11 QUERIES.

Q	Category	Naïve	Local	Global	Hybrid	Best
Q1	Multi-hop	P	P	P	P	P
Q2	Downstream	P	P	P	P	P
Q3	Path	P	P	P	P	P
Q4	Aggregation	P	F	P	F	P
Q5	Lookup	C	C	C	C	C
Q6	Temporal	C	C	C	C	C
Q7	What-If	P	P	P	C	C
Q8	SPOF	F	F	P	P	P
Q9	Inverse	F	F	F	F	F
Q10	Compare	P	P	F	P	P
Q11	Risk	F	F	F	F	F
Totals		2C	2C	2C	3C	3C

C = Correct, P = Partial, F = Fail.

H. Inter-Annotator Agreement

Cohen’s kappa coefficient was $\kappa = 0.716$, indicating *substantial agreement* per the Landis–Koch scale. Raw agreement was 16/22 decisions (73%). Intra-evaluator stability was 22/22 across three repeated runs, confirming deterministic evaluation at temperature 0.

Judge model circularity. The same model family (Claude)

serves as both generation engine and scoring judge. We mitigate this by providing explicit ground-truth answer sets and a structured rubric.

I. Threats to Validity

Internal validity. The 11 queries and their handlers were co-designed by the same author, creating a ceiling effect. The

TABLE XII
TYPED GRAPH PRIMITIVES EXPOSED TO THE LLM QUERY PLANNER.

Primitive	Operation
<code>find_nodes</code>	Scan by type + attribute filters
<code>get_node</code>	Single node attribute lookup
<code>get_neighbors</code>	1-hop with edge type filtering
<code>shortest_path</code>	Undirected shortest path
<code>subgraph</code>	Multi-hop BFS with direction
<code>count_edges</code>	In-/out-degree by edge type
<code>set_complement</code>	All nodes of type minus subset
<code>filter_edges_by_date</code>	Temporal edge filtering
<code>propagate_risk</code>	Weighted hop-distance scoring

core architectural argument does not depend on GraphRAG achieving a perfect score; it rests on the demonstrated failures of five independent LLM-based architectures.

External validity. The knowledge base is a single synthetic domain with 46 nodes. Scale testing confirms deterministic engine correctness at 1,100 nodes, but LLM-based architectures were only tested at 46 nodes.

Construct validity. Six construct threats merit attention: (1) Claude-as-judge circularity; (2) coarse 3-level scoring rubric; (3) single model family across all LLM architectures; (4) TF-IDF vs. dense embedding baseline; (5) template vs. LLM generation confound; (6) small evaluation set with purposive sampling.

Reproducibility. All source code, ground-truth answer sets, and benchmark harnesses are included. The deterministic core engine requires no API keys and produces identical output on every run.

VI. REVISION: GENERALIZED GRAPH QUERY PLANNING

The original evaluation demonstrated a clear failure gradient but was limited by co-design circularity. This section presents three methodological improvements.

A. Architecture 7: LLM Query Planner with Typed Graph Primitives

We introduce a seventh architecture that breaks the co-design circularity by replacing all 11 bespoke handlers with a single LLM-driven query planner. The planner receives the graph *schema* but not the data. Given a natural language query, the LLM emits `tool_use` calls selecting from nine typed graph primitives (Table XII).

B. Hold-Out Query Set

We constructed 12 hold-out queries covering all 10 intent categories plus 2 multi-category compositions. Ground truth was computed manually and verified with 13 automated validation tests.

C. Entity-Level F_1 Scoring

We replaced the coarse 3-level ordinal with entity-level precision, recall, and F_1 computed against ground-truth entity sets. Entity IDs are extracted via regex pattern matching and fuzzy entity name matching. The ordinal is retained as a secondary metric: $F_1 \geq 0.9 \rightarrow$ Correct, $0.3 \leq F_1 < 0.9 \rightarrow$ Partial, $F_1 < 0.3 \rightarrow$ Fail.

D. V2 Results

Table XIII presents the complete V2 results across four architectures and 23 queries, scored by entity-level F_1 .

E. Analysis

The LLM Query Planner outperforms all architectures. At $F_1 = 0.632$ (5C, 14P, 4F), it exceeds both the Agentic RAG ($F_1 = 0.577$) and the bespoke Deterministic GraphRAG ($F_1 = 0.472$). This inverts the V1 result.

The hold-out set reveals a generalization gap. The Deterministic GraphRAG drops from $F_1 = 0.574$ on original queries to $F_1 = 0.379$ on hold-out queries—a 34% relative decline—confirming co-design inflation. The LLM Planner shows the opposite: $0.557 \rightarrow 0.700$.

Typed primitives matter more than additional LLM reasoning steps. The Planner uses an average of 4.9 tool calls vs. 5.3 for the Agentic RAG, yet achieves higher F_1 . The difference is *better tools*, not more reasoning.

Remaining failure modes. Three categories remain challenging: aggregation (Q4, $F_1 = 0.23$), what-if (Q7, $F_1 = 0.10$), and subgraph comparison (Q10, $F_1 = 0.00$).

F. Revised Threats to Validity

The V2 revision addresses three of five primary threats: co-design circularity (Architecture 7 + hold-out set), construct validity (entity-level F_1), and statistical scope (11 $\rightarrow$ 23 queries). Two remain open: external validity (single synthetic domain) and cross-model replication (only Haiku 4.5 available).

VII. FROM TRAVERSAL TO COMPUTATION: ARCHITECTURE 8

Section VI established that typed traversal primitives outperform bespoke handlers. However, three categories remained intractable. We identified a common root cause: these queries require *computation over graph structure* rather than targeted traversal.

A. Graph Computation Primitives

We decomposed the three failure modes into specific computational capabilities (Table XIV).

Each computation tool encapsulates a complete graph algorithm—not a primitive step—so the LLM makes one tool call where Architecture 7 would require a multi-step loop.

B. Adaptive Tool Selection

Architecture 8 presents the LLM with all 15 tools (9 traversal + 6 computation) and a tool selection guide. The system prompt instructs: “For aggregation queries, use `aggregate_over_type` *INSTEAD* of manually iterating. For what-if queries, use `simulate_removal`. For comparison queries, use `subgraph_diff`.”

C. Results

Table XV presents Architecture 8 results compared with Architecture 7.

TABLE XIII
PER-QUERY ENTITY-LEVEL F_1 SCORES ACROSS FOUR ARCHITECTURES ON 23 QUERIES (11 ORIGINAL + 12 HOLD-OUT). ALL LLM-BASED ARCHITECTURES USE CLAUDE HAIKU 4.5. C = CORRECT ($F_1 \geq 0.9$), P = PARTIAL ($0.3 \leq F_1 < 0.9$), F = FAIL ($F_1 < 0.3$).

Query	Category	TF-IDF RAG	Det. GraphRAG	Agentic RAG	LLM Planner
<i>Original queries (co-designed with handlers)</i>					
Q1	Multi-hop	0.46	0.47	0.42	0.67
Q2	Downstream	0.38	0.91	0.81	0.91
Q3	Path	0.50	0.89	0.80	0.80
Q4	Aggregation	0.14	0.30	0.12	0.23
Q5	Simple Lookup	0.73	0.24	1.00	1.00
Q6	Temporal	0.44	0.80	0.25	0.50
Q7	What-If	0.25	0.40	0.33	0.10
Q8	SPOF	0.46	0.55	0.29	0.65
Q9	Inverse	0.00	0.53	0.36	0.42
Q10	Compare	0.00	0.36	0.14	0.00
Q11	Risk Heatmap	0.00	0.86	0.80	0.86
Original Mean		0.306	0.574	0.484	0.557
<i>Hold-out queries (unseen during development)</i>					
H1	Disruption	0.42	0.64	0.80	0.80
H2	Impact	0.26	0.87	0.74	0.80
H3	Path	0.53	0.67	0.80	0.77
H4	Temporal	0.60	0.29	1.00	1.00
H5	Aggregation	0.67	0.00	0.42	0.42
H6	Inverse	0.46	0.71	0.96	0.96
H7	SPOF (filtered)	0.29	0.47	0.63	0.95
H8	What-If	0.31	0.00	0.31	0.22
H9	Compare	0.62	0.33	0.48	0.38
H10	Risk Score	0.27	0.23	0.46	0.43
H11	Temp+Inverse	0.10	0.00	0.89	0.89
H12	Disr+Agg	0.13	0.34	0.46	0.78
Hold-out Mean		0.388	0.379	0.662	0.700
Overall Mean		0.349	0.472	0.577	0.632
Ordinal (C/P/F)		0/13/10	1/16/6	3/16/4	5/14/4

TABLE XIV
SIX GRAPH COMPUTATION PRIMITIVES ADDED IN ARCHITECTURE 8.

Primitive	Operation	Resolves
simulate_removal	Remove node; report cascade	What-If
subgraph_diff	BFS from two roots; diff	Compare
aggregate_over_type	Count reachable targets	Aggregation
betweenness	Betweenness centrality	Bottleneck
pagerank	PageRank importance scores	Influence
connected_comp.	Weakly connected components	Fragmentation

D. Analysis: The Measurement Gap

The headline F_1 numbers are flat ($0.635 \rightarrow 0.636$). This conceals a qualitative breakthrough. Consider Q4 (aggregation):

- **Architecture 7** calls `find_nodes` and `get_neighbors` repeatedly, running out of step budget.
- **Architecture 8** calls `find_nodes(Supplier)` once, then `simulate_removal` eight times. In 2 steps, it produces a *complete, correct ranking* of all 8 suppliers by product impact.

The answer is correct. Yet $F_1 = 0.22$ because the ground truth contains only 3 entity IDs while the comprehensive answer mentions 23 entities. This is a **fundamental limitation of entity-level F_1 for structural queries**. A correct aggregation answer *must* mention all entities in the ranking; the scorer counts additional entities as false positives.

E. Tool Adoption Patterns

Haiku 4.5 reliably selects the appropriate computation tool:

- **Q4:** Called `simulate_removal`×8 without instruction, producing a correct supplier-by-impact ranking.
- **Q10:** Called `subgraph_diff` then supplemented with targeted traversal.
- **Q7:** Did *not* adopt `simulate_removal` for dual-sourcing, suggesting counterfactual phrasing is harder for small models to map to removal tools.

The tool adoption rate was selective: the LLM chose computation tools only when the query category matched, correctly ignoring them for traversal-native queries.

F. The Operator Vocabulary Thesis

The progression across eight architectures recapitulates programming language evolution (Table XVI).

The thesis: **the barrier to graph reasoning is not the LLM’s intelligence—it is the operator vocabulary**. Architecture 7’s remaining failures were not failures of reasoning;

TABLE XV
ARCHITECTURE 8 VS. 7 ON 23 QUERIES (CLAUDE HAIKU 4.5). F_1
SCORES ARE ENTITY-LEVEL.

Query	Category	A7 F_1	A8 F_1	Δ	Tools Used
q1	Multi-hop	.615	.667	+0.05	—
q2	Downstream	.857	.857	.00	—
q3	Path	.800	.800	.00	—
q4	Aggregation	.211	.222	+0.01	sim_rem×8
q5	Lookup	1.00	1.00	.00	—
q6	Temporal	.500	.500	.00	—
q7	What-If	.250	.143	−.11	—
q8	SPOF	.647	.629	−.02	—
q9	Inverse	.421	.421	.00	—
q10	Compare	.080	.091	+0.01	sub_diff
q11	Risk	.857	.818	−.04	—
<i>Hold-out queries</i>					
h1	Disruption	.833	.800	−.03	—
h2	Impact	.769	.769	.00	—
h3	Path	.769	.769	.00	—
h4	Temporal	1.00	1.00	.00	—
h5	Aggregation	.417	.471	+0.05	—
h6	Inverse	1.00	.963	−.04	—
h7	SPOF	.947	.947	.00	—
h8	What-If	.667	.667	.00	—
h9	Compare	.375	.353	−.02	—
h10	Risk	.303	.323	+0.02	—
h11	Temp+Inv	.696	.762	+0.07	—
h12	Disr+Agg	.600	.667	+0.07	—
Mean		.635	.636	+0.001	
Ordinal		4C / 16P / 3F (both)			

TABLE XVI
THE RETRIEVAL-TO-COMPUTATION SPECTRUM.

Tier	Analogy	Arch.	Operator Vocabulary
Search	grep	1–2	Text similarity
Assembly	Hand-coded	3	Bespoke handlers
Macros	Reusable	6	4 generic tools
Typed	Typed ops	7	9 traversal primitives
Compiler	Alg. library	8	9 trav. + 6 comp.

the LLM correctly identified what it needed to do but lacked the tool. Architecture 8 supplies those tools, and the LLM adopts them without further instruction.

The practical implication: rather than engineering bespoke handlers or training graph-specialized models, practitioners should invest in *curating the right operator vocabulary*—a library of typed, composable graph operations exposed as LLM tools. When new query categories emerge, adding a tool (not a handler) extends capability.

VIII. CONCLUSION

We have presented an architectural comparison of eight retrieval systems for industrial supply chain intelligence, evaluated across 23 queries using entity-level F_1 scoring. The progression from flat text retrieval (Architectures 1–2) through bespoke graph handlers (Architecture 3) to LLM-composed traversal (Architecture 7) and computation (Architecture 8) reveals: **the limiting factor in graph-augmented retrieval is not the LLM’s reasoning capability but the operator vocabulary available to it.**

Architecture 7 demonstrated that typed traversal primitives outperform hand-coded handlers ($F_1 = 0.632$ vs. 0.472) while generalizing to unseen queries. Architecture 8 extends this: when computation tools are added, the LLM selectively adopts them for the exact query categories where traversal fails.

A critical methodological finding: **entity-level F_1 systematically underscores structural queries** where comprehensive answers are correct. This measurement gap suggests that structural query evaluation requires task-specific metrics—ranking accuracy for aggregation, structural completeness for comparison, causal coverage for what-if—rather than flat entity extraction.

Our taxonomy of six RAG failure modes (absence, degree, complement, topology, propagation, and temporal blindness) complements engineering-oriented failure taxonomies [20] by focusing on *structural* failure modes. Open challenges remain: cross-model replication, cross-domain validation, and developing evaluation metrics appropriate for structural queries. The reference implementation—including all eight architectures, 23 queries with ground truth, the F_1 scoring module, and the unified benchmark harness (8,154 lines across 17 source files)—is provided as a reproducible artifact (Appendix).

REFERENCES

- [1] J. Han, A. Cheung, Y. Wei, Z. Yu, X. Wang, B. Zhu, and Y. Yang, “RAG meets temporal graphs: Time-sensitive modeling and retrieval for evolving knowledge,” *arXiv preprint arXiv:2510.13590*, 2025.
- [2] Microsoft GraphRAG, “Incremental indexing: Design notes for graphrag.append,” *GitHub Issue Discussion*, microsoft/graphrag, 2024–2025.
- [3] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020.
- [4] D. Edge *et al.*, “From local to global: A graph RAG approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [5] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG: Simple and fast retrieval-augmented generation,” *arXiv preprint arXiv:2410.05779*, 2024.
- [6] B. J. Gutierrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Proc. NeurIPS*, 2024.
- [7] B. J. Gutiérrez, Y. Shu, W. Qi, S. Zhou, and Y. Su, “From RAG to memory: Non-parametric continual learning for large language models,” in *Proc. ICML*, 2025.
- [8] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proc. SciPy*, 2008.
- [9] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [11] S. Yan, J. Gu, Y. Zhu, and Z. Ling, “Corrective retrieval augmented generation,” *arXiv preprint arXiv:2401.15884*, 2024.
- [12] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *Proc. ICLR*, 2024.
- [13] X. He *et al.*, “G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering,” in *Proc. NeurIPS Workshop*, 2024.
- [14] K. Soman *et al.*, “Biomedical knowledge graph-enhanced prompt generation for large language models,” *arXiv preprint arXiv:2311.17330*, 2023.
- [15] D. Diefenbach, V. López, K. Singh, and P. Maret, “Core techniques of question answering systems over knowledge bases: A survey,” *Knowledge and Information Systems*, vol. 55, no. 3, pp. 529–569, 2018.
- [16] P. Cao, Y. Shi, J. Chen, S. Yu, and Y. Wang, “KQA Pro: A dataset with explicit compositional programs for complex question answering over knowledge base,” in *Proc. ACL*, 2022.

TABLE XVII

SOURCE FILE INVENTORY. CORE ENGINE FILES REQUIRE ZERO API KEYS.

File	Lines	Role
<i>Core Engine (zero API keys)</i>		
data.py	524	Knowledge base + ground truth
graphrag_engine.py	1,709	11 handlers, 5 mutations
rag_engine.py	179	TF-IDF top-K retrieval
app.py	118	Flask web server
static/app.js	625	Frontend: vis.js + controls
static/style.css	1,168	UI styling
templates/index.html	355	Jinja2 template
<i>V2 Revision (Section VI)</i>		
graph_primitives.py	530	9 typed primitives + 15 tests
graph_query_planner.py	250	Architecture 7: LLM planner
holdout_queries.py	310	12 hold-out queries
scoring.py	200	Entity-level F_1 scoring
benchmark_runner.py	280	Unified benchmark harness
<i>V3 Revision (Section VII)</i>		
graph_computation.py	630	6 computation primitives
adaptive_planner.py	594	Architecture 8
<i>Benchmark Scripts (Anthropic API)</i>		
llm_graphrag_bench.py	482	LLM-Based GraphRAG
agentic_rag.py	456	ReAct agentic baseline
lightrag_benchmark.py	342	LightRAG (HKU) benchmark
dense_rag_benchmark.py	264	Dense-embedding baseline
scale_test.py	421	Scalability benchmark
inter_annotator.py	205	Cohen’s κ computation
Total	6,848	

- [17] J. Sun *et al.*, “Think-on-Graph: Deep and responsible reasoning of large language model on knowledge graph,” in *Proc. ICLR*, 2024.
- [18] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” in *Proc. EACL System Demonstrations*, 2024.
- [19] J. Jiang *et al.*, “StructGPT: A general framework for large language model to reason over structured data,” in *Proc. EMNLP*, 2023.
- [20] S. Barnett *et al.*, “Seven failure points when engineering a retrieval augmented generation system,” *arXiv preprint arXiv:2401.05856*, 2024.

APPENDIX A

IMPLEMENTATION ARTIFACT SUMMARY

TABLE XVIII
GROUND-TRUTH ANSWER SETS FOR INDEPENDENT VERIFICATION OF TABLE V CORRECTNESS ASSESSMENTS.

ID	Category	Ground-Truth Answer Set	RAG Scoring Rationale
Q1	Multi-hop	Thailand flood (EVT-001) → ThaiRubber Co (SUP-004) → CMP-004, CMP-011 → FAC-005 . 3 hops.	Fail — cannot trace supplier→component→factory.
Q2	Downstream	TechChip (SUP-001) → CMP-001, -006, -014 → FAC-001, -004 → PRD-001, -002, -004, -005, -006 → all 4 customers. 4 hops.	Fail — cannot trace full 4-hop cascade.
Q3	Path	SUP-002 $\xrightarrow{\text{SUPPLIES}}$ CMP-002 $\xleftarrow{\text{USES}}$ FAC-001 $\xrightarrow{\text{PRODUCES}}$ PRD-001. 3 hops.	Fail — cannot construct a path.
Q4	Aggregation	TechChip (SUP-001) = 5 products, ElectraWire (SUP-008) = 5 (tied).	Fail — requires full graph traversal.
Q5	Lookup	AeroMetal Corp (SUP-002) supplies: CMP-002, CMP-009, CMP-015.	Partial — may miss components.
Q6	Temporal	Current: TechChip Inc (since 2024-02-17). Expired: ShenzenChip (ended 2024-02-16).	Partial — retrieves both without distinguishing.
Q7	What-If	CMP-001 type = Electronic Assembly. Sole supplier: TechChip. No alternatives in graph.	Fail — identifying structural absence.
Q8	SPOF	15/15 components (100%) single-supplier. High criticality: 11; Medium: 3; Low: 1.	Fail — requires SUPPLIES in-degree for every node.
Q9	Inverse	All customers: CUS-001–004. Affected: CUS-001–003. Unaffected: CUS-004 (DefenseTech) .	Fail — requires universal set + blast radius.
Q10	Compare	WideBird: 8 suppliers, 12 components, 4 factories. RegionalJet: 2, 4, 1. Shared: TechChip, ElectraWire.	Fail — requires dual upstream traversal.
Q11	Risk	Eq. (1): WideBird=1.19, RegionalJet=ExecWing=SkyPatrol=0.98, CargoHawk=0.70, NarrowBody=0.49.	Fail — weighted multi-hop is structural.

TABLE XIX
DEPENDENCIES.

Package	Version	Purpose
<i>Core Engine (offline)</i>		
flask	≥ 3.0	Web server
networkx	≥ 3.0	Graph algorithms
scikit-learn	≥ 1.3	TF-IDF, cosine sim.
numpy	≥ 1.24	Numerical operations
vis-network	9.1.6 (CDN)	Graph visualization
<i>Benchmark Scripts</i>		
anthropic	≥ 0.40	Claude API client
lightrag-hku	≥ 1.4	LightRAG benchmark
sentence-trans.	≥ 3.0	Dense embeddings